

Yes — I've got it. This changes the architecture from "Prompt Refinery helps the user talk to Codex" into **Prompt Refinery as the broker/gatekeeper between a coding agent and a reviewer/admin layer.**

Thought for 25s >

Yes. Got it.

This is the sharper architecture:

```
Prompt Refinery is not just a prompt builder.
```



```
Prompt Refinery is the middleman / broker / gatekeeper between:
```

1. Codex or coding AI
2. Reviewer AI and/or human admin

The coding agent should **not** be used for back-and-forth clarification. Codex is the expensive executor. The reviewer/admin side is where questions, critiques, scope checks, thread decisions, and prompt refinement happen.

That is the correct design.

Revised core product definition

```
Prompt Refinery MVP = local-only manual broker loop between Codex and Reviewer/Admin
```



The loop is not:

```
User ↔ Codex ↔ User
```



The loop is:

```
Codex output
→ Reviewer AI evaluates it
→ Reviewer suggests next Codex prompt
→ Prompt Refinery audits/refines that suggested prompt
→ Human/admin or reviewer approves
→ Accepted prompt goes back to Codex
→ Codex outputs work + suggested next prompt
→ repeat
```



Prompt Refinery is the **acceptance-handshake layer** before anything goes back to Codex.

The real architecture

Actors

Actor	Job
Codex / Coding AI	Executes coding/build tasks. Should receive only polished, minimal, accepted prompts.
Reviewer AI	Reviews Codex output, analyzes scope, critiques next prompt, proposes next action.
Prompt Refinery	Middleman that validates, compresses, critiques, reconciles, and formats the next prompt.
Human Admin	Final override authority when needed. Can accept/reject reviewer decisions.
File Writer API	Saves modes, loops, summaries, specs, passes, context packs, decisions.
Context Store	Holds summaries, files, image descriptions, accepted prompts, phase/spec status.

The middleman rule

This is the most important rule:

No unnecessary back-and-forth with Codex.



Codex should not be asked:

Can you clarify?
What did you mean?
Should we do X or Y?
Do you agree with this critique?



Those questions go to:

Reviewer AI
Human admin
Prompt Refinery decision engine



Codex only receives:

```
Accepted, compressed, context-aware, mode-formatted next prompt.
```



That is how you protect rate limits and token usage.

The two directions of the loop

Direction A — Codex to Reviewer/Admin through Prompt Refinery

Codex returns:

1. Work completed
2. Files changed
3. Issues found
4. Suggested next prompt
5. Risks / assumptions



Then Prompt Refinery receives the full Codex output and does:

1. Understand what Codex did.
2. Extract Codex's suggested next prompt.
3. Summarize the pass.
4. Detect changed scope.
5. Detect whether the next prompt is good, wasteful, vague, risky, or too broad.
6. Package everything for Reviewer AI or Human Admin.



Then Reviewer AI returns:

1. Review of Codex output
2. Agreement/disagreement with Codex's suggested next prompt
3. Suggested prompt to actually send back to Codex
4. Questions or critique items
5. Recommended thread decision



Then Prompt Refinery audits the reviewer's recommendation.

Direction B — Reviewer/Admin to Codex through Prompt Refinery

Reviewer suggests a prompt.

Prompt Refinery does **not** blindly pass it to Codex.

Prompt Refinery checks:

1. Does the reviewer understand Codex's output?
2. Does the suggested prompt match the actual thread goal?
3. Is the prompt too big?
4. Is the prompt missing context?
5. Is the prompt asking Codex to decide things the reviewer/admin should decide?
6. Is this still within the phase/spec/MVP scope?
7. Should the thread close first?
8. Can the prompt be shorter?
9. Does the prompt need wrappers?
10. Does the prompt need a stricter stop condition?



Then Prompt Refinery produces:

Accepted prompt to give Codex



That is the handshake.

Acceptance handshake

This should become a named product feature:

Acceptance Handshake



The handshake happens before any prompt goes to Codex.

Handshake checklist

Reviewer understood Codex output
Reviewer understood Codex's suggested next prompt
Prompt Refinery understood both
Prompt Refinery critiqued both
Unresolved questions handled by reviewer/admin
Scope checked against current thread/phase/spec
Token load estimated
Thread close decision made
Prompt compressed
Wrappers applied
Mode/family/template applied
Human/admin approval checked if required



Only then:

Send accepted prompt to Codex.



Admin model

You are right: admin can be the reviewer AI, the human, or both.

Admin settings

YAML



```
admin_policy:
  reviewer_ai_can_accept:
    low_risk_prompt_changes: true
    thread_close_recommendations: true
    scope_changes: false
    file_deletion_or_destructive_changes: false

  human_must_accept:
    phase_change: true
    spec_change: true
    destructive_action: true
    large_token_spend: true
    external_connector_call: true
    template_permanent_update: true

  dual_accept_required:
    mvp_scope_change: true
    webhook_or_api_connector: true
    auto_loop_enablement: true
```

For MVP:

```
Human admin has final authority.
Reviewer AI can recommend.
Prompt Refinery can enforce rules.
```



The hierarchy model

This part you said is important, and you are right.

The app should organize work like this:

```
Product
  → Spec / Version
    → Phase
      → Feature
        → Thread
          → Pass
            → Prompt
```



→ Output
→ Review
→ Accepted next prompt

Meaning

Level	Meaning
Product	Prompt Refinery itself, or any project being built
Spec / Version	v0/MVP, v1, v2, etc.
Phase	A major build chunk inside a spec
Feature	A user-facing or technical feature
Thread	One contained work conversation for a feature/scope
Pass	One Codex execution + review + next prompt cycle
Prompt	The accepted prompt sent to Codex
Output	Codex's response
Review	Reviewer/admin analysis
Accepted next prompt	Refined prompt approved for next pass

This gives the product real project memory.

Version language

Use this:

```
Spec 0 = v0 = MVP  
Spec 1 = v1  
Spec 2 = v2  
Spec 3 = v3
```



That should be built into the product model.

Example:

```
</> YAML
```



```
product_id: prompt_refinery
current_spec: spec_0
version_label: v0_mvp
phase: phase_4_manual_loop_runner
feature: codex_manual_loop
thread: thread_003_mode_manager_integration
pass: pass_007
```

Thread-start policy

The reviewer should decide when to start a new thread based on a configurable policy.

You named the right levels:

```
lazy
reasonable
frequent
aggressive
```



Thread renewal policies

Policy	Behavior
Lazy	Keep thread going unless context is obviously bloated or scope changed hard.
Reasonable	Suggest new thread when feature/scope shifts or token estimate gets high.
Frequent	Start new thread after every major subtask or 3–5 passes.
Aggressive	Start new thread often; use summaries to minimize token load.

For MVP, default should be:

```
reasonable
```



Because “lazy” risks token bloat, and “aggressive” creates too much workflow friction.

Thread close criteria

```
Start new thread if:
- Pass count exceeds policy threshold
- Scope changed
- Phase changed
```



- Feature changed
- Codex output is too long
- Context estimate is high
- Reviewer detects drift
- Current next prompt no longer depends on full previous thread

Revised MVP flow

This is the exact MVP loop:

1. Human writes intent.
2. Prompt Refinery creates minimal Codex prompt.
3. Human copies prompt to Codex.
4. Codex returns output + suggested next prompt.
5. Human pastes Codex output into Prompt Refinery.
6. Prompt Refinery extracts Codex's suggested next prompt.
7. Prompt Refinery prepares a review packet.
8. Reviewer AI or human admin reviews the packet.
9. Reviewer suggests the prompt that should actually go back to Codex.
10. Prompt Refinery critiques reviewer suggestion.
11. Prompt Refinery asks reviewer/admin questions only if needed.
12. Prompt Refinery resolves or defers issues.
13. Prompt Refinery applies mode/family/wrappers/token policy.
14. Prompt Refinery creates accepted next prompt.
15. Human copies accepted prompt to Codex.
16. Repeat.

Notice what is missing:

Prompt Refinery does not ask Codex clarifying questions.

Good. That is the right constraint.

Revised MVP definition

MVP = local-only manual Codex/Reviewer broker loop with acceptance handshake, pass

MVP must include

Mode Manager
Codex Manual Broker Mode


```
Reviewer Packet Mode
Acceptance Handshake
Manual pass history
Prompt handoff objects
Reviewer/admin decision states
Text-file context
Manual image-description context
Thread summary
Thread close recommendation
Estimated token load
Scope/phase/spec tracker
Prompt wrappers
Template registry
File-writer persistence
```

MVP must not include yet

```
Automatic Codex API execution
OpenRouter calls
n8n calls
Make/Zapier calls
Raw video parsing
Live web research mode
Full MCP implementation
Cloud sync
Multi-user auth
Template marketplace
```



The uploaded research/web-augmentation notes still matter, but that should plug in later as a mode that checks templates against sources; it should not be part of the first local-only manual broker MVP. 

Research augmented prompt mode

Core mode: Codex Reviewer Broker Mode

This is the real built-in mode.

```
Codex Reviewer Broker Mode
```



Mode template

```
</> Markdown
```



```

---
mode_id: codex_reviewer_broker
display_name: Codex Reviewer Broker
version: 1
status: built_in
mutable: false
copyable: true
default_prompt_family: execution_plan
token_policy: minimal
scope_policy: exact_task_only
question_policy: ask_reviewer_or_admin_only
critique_policy: required_before_codex_handoff
context_policy: selected_context_plus_thread_summary
loop_policy: manual_broker_loop
connector_policy: manual_copy
thread_policy: reasonable
admin_policy: human_final_approval
---

# Codex Reviewer Broker Mode

## Purpose

Act as the middleman between Codex and Reviewer/Admin. Never send raw reviewer suggestion

## Operating rules

1. Codex is the executor, not the discussion partner.
2. Reviewer/Admin is the clarification partner.
3. Prompt Refinery must understand Codex output before accepting a next prompt.
4. Prompt Refinery must understand the reviewer's reasoning before accepting a next prompt.
5. Prompt Refinery must critique both Codex's suggested next prompt and the reviewer's suggestion.
6. Prompt Refinery may ask reviewer/admin questions.
7. Prompt Refinery should avoid asking Codex questions.
8. Every Codex handoff requires an acceptance handshake.
9. Every pass should update thread/pass/spec state.
10. Every pass should include token estimate and thread-close recommendation.

## Required output

Return:

1. Pass summary
2. Codex output understanding
3. Codex suggested next prompt extraction
4. Reviewer/admin recommendation summary
5. Prompt Refinery critique
6. Open questions for reviewer/admin
7. Scope/thread decision
8. Token estimate

```

9. Accepted next prompt for Codex
10. Handshake status

Handoff object

This should become a saved file type.

 YAML 

```
handoff_id: handoff_0007
product_id: prompt_refinery
spec_id: spec_0_mvp
phase_id: phase_4_manual_broker_loop
feature_id: codex_reviewer_broker
thread_id: thread_003
pass_id: pass_007

from_actor: reviewer_ai
to_actor: codex
broker: prompt_refinery

codex_previous_output:
  summary: ""
  raw_output_path: ""

codex_suggested_next_prompt:
  raw: ""
  critique: ""

reviewer_recommendation:
  summary: ""
  suggested_prompt_to_codex: ""
  confidence: medium
  admin_decisions:
    - decision: ""

prompt_refinery_decision:
  accepted: false
  changes_made:
    - ""
  rejected_parts:
    - ""
  questions_to_reviewer_or_admin:
    - ""
  final_prompt_to_codex: ""

acceptance_handshake:
  scope_checked: false
  token_checked: false
```

```
reviewer_aligned: false
admin_approved: false
ready_for_codex: false
```

Pass object

Every pass should be saved.

YAML



```
pass_id: pass_004
thread_id: thread_002
phase_id: phase_1_mode_manager
spec_id: spec_0_mvp

pass_type: codex_execution_review
status: accepted_next_prompt_ready

inputs:
  accepted_prompt_sent_to_codex: ""
  context_pack_id: ctx_003
  active_mode_id: codex_reviewer_broker
  active_family: execution_plan
  wrappers:
    - minimal_token_wrapper
    - codex_task_wrapper
    - acceptance_handshake_wrapper

outputs:
  codex_raw_output: ""
  codex_suggested_next_prompt: ""
  reviewer_raw_review: ""
  reviewer_suggested_next_prompt: ""
  prompt_refinery_final_prompt: ""

decisions:
  scope_decision: continue_same_thread
  thread_decision: continue
  admin_decision: approved
  critique_decision: fix_now

token_estimate:
  prompt_to_codex: 0
  codex_output: 0
  reviewer_packet: 0
  final_next_prompt: 0
  total_context_estimate: 0
```

Reviewer packet

Prompt Refinery should create a packet for the reviewer.

</> Markdown



```
# Reviewer Packet
```

```
## Current Product Position
```

```
- Product:
- Spec/version:
- Phase:
- Feature:
- Thread:
- Pass:
```

```
## Prompt Sent to Codex
```

```
```text
```

```
...
```

## Codex Output

...



## Codex Suggested Next Prompt

...



## Prompt Refinery Initial Understanding

- What Codex did:
- What changed:
- What remains:
- Risks:
- Scope drift:
- Missing context:

## Reviewer Task

Review the Codex output and suggested next prompt.

Return:

1. Whether Codex completed the requested work
2. Whether Codex's suggested next prompt is useful

3. What the next prompt to Codex should actually be
4. What should be fixed now vs deferred
5. Whether to continue this thread or start a new one
6. Whether human admin approval is required



```

Prompt Refinery audit of reviewer

After reviewer responds, Prompt Refinery does this:

```markdown id="kh3qux"
# Reviewer Recommendation Audit

## Inputs

- Codex output
- Codex suggested next prompt
- Reviewer review
- Reviewer suggested next prompt
- Current scope/spec/phase/thread
- Token policy
- Active mode/family/wrappers

## Check

1. Did reviewer understand Codex output?
2. Did reviewer correctly evaluate Codex's suggested prompt?
3. Did reviewer introduce scope creep?
4. Did reviewer miss cheaper next action?
5. Did reviewer ask Codex to make admin decisions?
6. Should any question go to reviewer/human instead?
7. Should this thread close?
8. Can the prompt be compressed?
9. Does this require human admin approval?

## Output

- Approved reviewer points
- Rejected reviewer points
- Questions to reviewer/admin
- Final accepted Codex prompt
- Handshake status
```

Families needed for this architecture

The prompt library should have these families.

```
codex_broker
reviewer_packet
acceptance_handshake
prompt_handoff
thread_summary
token_budget
context_pack
scope_control
phase_planning
spec_management
execution_plan
debugging_troubleshooting
critique_loop
decision_memo
repo_audit
code_review
template_gap_analysis
```



Modes needed

```
codex_reviewer_broker
minimal_token_efficiency
strict_mvp_scope
reviewer_admin_mode
human_admin_mode
dual_admin_mode
thread_lazy_mode
thread_reasonable_mode
thread_frequent_mode
thread_aggressive_mode
acceptance_handshake_required
context_compression_mode
codex_output_understanding_mode
next_prompt_refinement_mode
```



Wrappers needed

Wrappers format and constrain prompts.

```
codex_task_wrapper
reviewer_packet_wrapper
accepted_prompt_wrapper
minimal_token_wrapper
scope_guard_wrapper
thread_summary_wrapper
```



```
handoff_wrapper
question_to_reviewer_wrapper
question_to_human_admin_wrapper
critique_decision_wrapper
next_prompt_wrapper
file_context_wrapper
image_description_context_wrapper
token_budget_wrapper
stop_condition_wrapper
acceptance_handshake_wrapper
```

Skills needed

Skills are reusable operations.

```
extract_codex_suggested_next_prompt
summarize_codex_output
understand_codex_output
critique_codex_output
critique_codex_next_prompt
build_reviewer_packet
audit_reviewer_recommendation
ask_reviewer_question
ask_human_admin_question
resolve_admin_decision
compress_prompt_for_codex
estimate_token_load
decide_thread_policy
generate_thread_summary
generate_continuation_prompt
apply_wrappers
perform_acceptance_handshake
save_pass
save_handoff
save_context_pack
save_spec_state
```



Prompt templates to add

```
prompt_library/codex_broker/
```

```
codex_initial_prompt_generator_v1.md
codex_output_understanding_v1.md
codex_suggested_next_prompt_extractor_v1.md
```



codex_next_prompt_refiner_v1.md
codex_final_handoff_prompt_v1.md

prompt_library/reviewer_packet/

reviewer_packet_v1.md
reviewer_response_contract_v1.md
reviewer_scope_decision_v1.md
reviewer_thread_policy_decision_v1.md



prompt_library/acceptance_handshake/

acceptance_handshake_v1.md
handoff_readiness_check_v1.md
admin_approval_check_v1.md
reviewer_alignment_check_v1.md



prompt_library/prompt_handoff/

handoff_object_v1.md
codex_to_reviewer_handoff_v1.md
reviewer_to_codex_handoff_v1.md
manual_copy_handoff_v1.md



prompt_library/thread_summary/

close_thread_summary_v1.md
continuation_summary_v1.md
thread_scope_drift_check_v1.md
start_new_thread_recommendation_v1.md



prompt_library/token_budget/

token_estimate_v1.md
token_saver_recommendation_v1.md
context_compression_decision_v1.md
minimal_prompt_rewrite_v1.md



prompt_library/context_pack/

manual_context_pack_v1.md
text_file_context_v1.md
image_description_context_v1.md
thread_summary_context_v1.md
included_excluded_context_v1.md



prompt_library/scope_control/

```
strict_mvp_scope_check_v1.md
phase_scope_check_v1.md
feature_scope_check_v1.md
scope_creep_detector_v1.md
defer_or_fix_now_v1.md
```



prompt_library/spec_management/

```
spec_0_mvp_tracker_v1.md
spec_version_plan_v1.md
phase_breakdown_v1.md
feature_completion_check_v1.md
```



MVP phases, revised again

This is now the cleanest build plan.

Phase 0 — Product model foundation

Goal:

Add product/spec/phase/thread/pass concepts.



Passes:

Pass	What it does
0.1	Add <code>ProductState</code> type
0.2	Add <code>SpecState</code> type
0.3	Add <code>PhaseState</code> type
0.4	Add <code>ThreadState</code> type
0.5	Add <code>PassState</code> type
0.6	Save/load state through file-writer API

Done when:

The app knows which spec, phase, thread, and pass the user is in.



Phase 1 — Mode Manager integration

Goal:

Make broker mode selectable and real.



Passes:

Pass	What it does
1.1	Mount Mode Manager
1.2	Add <code>codex_reviewer_broker</code> built-in mode
1.3	Add thread policy modes
1.4	Add admin policy setting
1.5	Save custom modes through file-writer API
1.6	Load custom modes on app start

Done when:

User can select Codex Reviewer Broker Mode and choose admin/thread policy.



Phase 2 — Broker template registry

Goal:

Create all templates needed for handoff, review, acceptance, token, context, and score.



Passes:

Pass	What it does
2.1	Add <code>codex_broker/</code> templates
2.2	Add <code>reviewer_packet/</code> templates
2.3	Add <code>acceptance_handshake/</code> templates
2.4	Add <code>prompt_handoff/</code> templates

Pass	What it does
2.5	Add <code>thread_summary/</code> templates
2.6	Add <code>token_budget/</code> templates
2.7	Add <code>scope_control/</code> templates
2.8	Register templates in app metadata

Done when:

The app can show which template controls each step of the broker loop.



Phase 3 — Context Pack MVP

Goal:

Control what context gets included in reviewer packets and Codex prompts.



Passes:

Pass	What it does
3.1	Add Context Pack panel
3.2	Add manual notes
3.3	Add text-file context
3.4	Add manual image-description context
3.5	Add thread summary context
3.6	Add included/excluded context preview
3.7	Save context packs through file-writer API

Done when:

Prompt Refinery can produce a Codex prompt or reviewer packet with explicit include



Phase 4 — Manual broker loop runner

Goal:

Build the actual Codex ↔ Reviewer/Admin middleman workflow.



Passes:

Pass	What it does
4.1	Create loop session UI
4.2	Create pass history UI
4.3	Generate accepted prompt to Codex
4.4	Paste Codex output
4.5	Extract Codex suggested next prompt
4.6	Build reviewer packet
4.7	Paste reviewer output
4.8	Extract reviewer suggested next prompt
4.9	Audit reviewer suggestion
4.10	Generate final accepted Codex prompt
4.11	Require acceptance handshake before copy
4.12	Save handoff/pass objects

Done when:

A full Codex → Reviewer → Prompt Refinery → Codex cycle can happen manually.



This is the main MVP feature.

Phase 5 — Decision popups

Goal:

Questions and critiques are resolved with reviewer/admin, not Codex.



Passes:

Pass	What it does
5.1	Add question popup
5.2	Add target: reviewer / human admin / both
5.3	Add accept / reject / defer / fix now
5.4	Add reviewer decision audit
5.5	Add unresolved decision tracker
5.6	Block handoff if required decisions are unresolved

Done when:

Prompt Refinery can hold a short decision dialogue with Reviewer/Admin before Codex



Phase 6 — Thread management and token saver

Goal:

Prevent thread bloat and control pass/thread/phase scope.



Passes:

Pass	What it does
6.1	Add token estimate panel
6.2	Add pass-count tracking
6.3	Add thread policy selector: lazy/reasonable/frequent/aggressive
6.4	Add scope drift detector
6.5	Add close-thread summary
6.6	Add continuation prompt

Pass	What it does
6.7	Add start-new-thread recommendation
6.8	Save summaries through file-writer API

Done when:

Reviewer/Admin can decide when to close threads, start new ones, and continue from



Phase 7 — MVP hardening

Goal:

Make v0 usable as a real local product.



Passes:

Pass	What it does
7.1	Add copy buttons for every handoff
7.2	Add local save/load reliability
7.3	Add export loop report
7.4	Add template preview
7.5	Add error states
7.6	Add README workflow
7.7	Add QA checklist

Done when:

You can use the app for a real Codex project from intent to multiple reviewed passes



After Phase 7, MVP/spec 0/v0 is done.

MVP acceptance test

The MVP is complete when this works:

1. User opens Prompt Refinery.
2. User selects Codex Reviewer Broker Mode.
3. User selects thread policy: reasonable.
4. User writes human intent.
5. Prompt Refinery creates accepted prompt to Codex.
6. User copies prompt to Codex.
7. User pastes Codex output back into Prompt Refinery.
8. Prompt Refinery extracts Codex's suggested next prompt.
9. Prompt Refinery builds reviewer packet.
10. User gives reviewer packet to Reviewer AI or human admin.
11. User pastes reviewer response into Prompt Refinery.
12. Prompt Refinery audits reviewer response.
13. Prompt Refinery asks reviewer/admin questions if needed.
14. Prompt Refinery resolves decisions.
15. Prompt Refinery applies wrappers and token policy.
16. Prompt Refinery creates accepted next prompt.
17. User copies accepted next prompt to Codex.
18. Pass is saved.
19. Thread policy recommends continue or close.
20. User can close thread and continue from summary.



That is the real MVP.

Post-MVP versions

Spec 1 / v1 — Local reviewer support

Ollama local reviewer
Local model selector
Local critique generation
Local summary generation
Local token compression



Still local-first.

Spec 2 / v2 — Provider connectors

OpenRouter connector
OpenAI-compatible connector
Provider budget settings
Provider model profiles



Manual approval before provider call
Provider logs

Spec 3 / v3 — Webhook automations

Generic webhook connector
n8n preset
Make preset
Zapier preset
Webhook payload templates
Webhook response parser
Manual approval before webhook call



Spec 4 / v4 — Research-augmented prompt validation

Search before prompt improvement
Source-backed template gap analysis
Best-practice lookup
Template tweak suggestions
Apply to run only
Suggest permanent template update



Spec 5 / v5 — Semi-auto broker loop

Prompt Refinery sends prompts to model/provider
Receives output
Routes to reviewer
Runs handshake
Still asks human before Codex handoff



Spec 6 / v6 — MCP-style system

Resources
Prompts
Tools
Roots
Elicitation
Permission gates
Connector registry



Final distilled version

You are building this:

A local-first manual AI broker that sits between Codex and Reviewer/Admin, turns ev



That is the product.

And yes — **middlemanning is the name of the game.**

But the more precise product word is:

broker



Prompt Refinery is the **AI prompt broker** between executor and reviewer.